

Lists

1. What are Lists ?

- Lists allow us to store multiple items in a single variable.

They are :

- ordered (maintain the order of elements),
- mutable (items can be changed after creation),
- allow duplicate values,
- dynamic (can store elements of various data types and other sequence types).

1.1 Creating a List

We create a list by placing elements inside square brackets, separated by commas.

Syntax

```
Python
list_name = [item1, item2, item3]
```

Example

```
Python
mixed_list = ["apple", 25, 3.14, True]
print(mixed_list)
```

Output

```
['apple', 25, 3.14, True]
```

1.2 Finding List Length: len()

To find the total number of elements in a list, we use the built-in len() function.

Syntax

```
Python
len(list_name)
```

Example

```
Python
num= [20, 25, 30, 35]
print(len(num))
```

Output

```
4
```

2. Accessing and Slicing Lists

2.1 Positive Indexing

- Every item in a list has an index starting from 0.
- We use indexing to access specific elements.

Syntax

```
Python
list_name[index]
```

Example

```
Python
languages = ['Telugu', 'Hindi', 'English']
print(languages[0])
```

Output

```
Telugu
```

2.2 Negative Indexing

- Negative indexing retrieves items starting from the end.
- Where -1 represents the very last element.

Syntax

```
Python  
list_name[-index]
```

Example

```
Python  
greetings= ['Hi', 'Hello', 'Hola']  
print(greetings[-1])
```

Output

```
Hola
```

2.3 Slicing a List

- We can access a part of a list using the colon operator
- where the starting index is included but the ending index is not.

Syntax

```
Python  
list_name[start:end]
```

Example

```
Python  
letter = ['a', 'b', 'c', 'd', 'e']  
print(letter[1:4])
```

Output

```
['b', 'c', 'd']
```

3. Modifying Lists

3.1 Changing List Items by Index

We can change the items of a list by accessing the index and assigning a new value using the equals operator.

Syntax

```
Python  
list_name[index] = new_value
```

Example

```
Python  
numbers = [1, 2, 3, 5]  
numbers[0] = 25  
print(numbers)
```

Output

```
[25, 2, 3, 5]
```

3.2 The append() Method

This method is used to add a single element directly to the end of a list.

Syntax

```
Python  
list_name.append(element)
```

Example

```
Python
a = [1, 2]
a.append(8)
print(a)
```

Output

```
[1, 2, 8]
```

3.3 The insert() Method

This allows adding an element anywhere in the list by specifying the exact index position.

Syntax

```
Python
list_name.insert(index, element)
```

Example

```
Python
fruits = ["banana", "cherry"]
fruits.insert(1, "apple")
print(fruits)
```

Output

```
['banana', 'apple', 'cherry']
```

3.4 The extend() Method

The extend method takes an iterable like another list and appends each of its elements to the existing list.

Syntax

```
Python  
list_name.extend(iterable)
```

Example

```
Python  
a = [1, 2]  
n = ['One', 'Two']  
a.extend(n)  
print(a)
```

Output

```
[1, 2, 'One', 'Two']
```

3.5 Modifying with enumerate()

A for loop with `enumerate()` allows us to iterate through a list while tracking the index positions to modify elements in place.

- **`enumerate(iterable, start)`**: a built in function that iterates through the iterable and assigns a counter to it.
 - returns an enumerate object.
 - the object stores a sequence of tuples with index and the value(index,value).
 - you need to convert it into list/tuple using `list()`, `tuple()`.
 - allows us to track loop index and value simultaneously.
 - Default start is 0.

Syntax

```
Python  
for index, value in enumerate(list_name):
```

Example

Python

```
# add 5 to the even numbers in the list
a = [1, 2, 3, 4]
for i, x in enumerate(a):
    if x % 2 != 0:
        a[i] = a[i] + 5
print(a)
```

Output

```
[6, 2, 8, 4]
```

3.6 Modifying with map()

- **map(function, iterable):** a built-in function that applies the function to each and every element of the iterable
 - returns a map object
 - we convert it into the list/set/tuple

Syntax

Python

```
list(map(function, list_name))
```

Example

Python

```
# add 10 to each element
a = [1, 2, 3]
a = list(map(lambda x: x + 10, a))
print(a)
```

Output

```
[12, 13, 14]
```

4. Removing Items from Lists

4.1 The remove() Method

This method deletes the very first occurrence of a specified value in the list.

Syntax

```
Python  
list_name.remove(value)
```

Example

```
Python  
level = ['easy', 'hard', 'medium']  
level.remove('hard')  
print(level)
```

Output

```
['easy', 'medium']
```

4.2 The pop() Method

This removes an element based on its index and returns the removed element to you.

Syntax

```
Python  
list_name.pop(index)
```

Example

Python

```
letters = ['P', 'S', 'C']  
removed_item = letters.pop(1)  
print(letters)  
print(removed_item)
```

Output

```
['P', 'C']  
S
```

4.3 The del Statement

- This statement can remove elements by specifying their exact index or a range of indices.
- If we try to access the element or list after deleting it shows error

Syntax

Python

```
del list_name[index]
```

Example

Python

```
marks = [23, 45, 56]  
del marks[0]  
print(marks)
```

Output

```
[45, 56]
```

4.4 The clear() Method

This empties the list directly by removing all elements from the list object entirely.

Syntax

```
Python  
list_name.clear()
```

Example

```
Python  
a = [1, 2, 3]  
a.clear()  
print(a)
```

Output

```
[]
```

5. Advanced List Operations

5.1 Iterating Through a List

We can use a simple for loop to iterate over the elements of a list one by one.

Syntax

```
Python  
for item in list_name:  
    # Code to execute
```

Example

```
Python  
colors = ['Red', 'Blue', 'Green']
```

```
for color in colors:  
    print(color)
```

Output

```
Red  
Blue  
Green
```

5.2 The index() Method

Returns the exact index position of the first matched item found in the list.

Syntax

```
Python  
list_name.index(value)
```

Example

```
Python  
a = [10, 20, 30]  
print(a.index(30))
```

Output

```
2
```